

yah.mobile — Manus/Firebase 並行運用&モバイルアプリ展開 アーキテクチャ レポート

対象システム: yah.mobile (訪日外客向け eSIM 販売プラットフォーム) **作成日:** 2026年6月27日 **改訂:** 2026年6月27日 (モバイルアプリ展開の視点を追加) **改訂:** 2026年6月27日 (メール送信を Resend 確定採用に更新) **改訂:** 2026年6月27日 (月25,000件購買規模を前提にスケール設計章を追加) **改訂:** 2026年6月27日 (chat.yah.mobi の Firebase 移行と yah.mobi 連携設計を追加) **改訂:** 2026年6月27日 (GCP プロジェクト構成と magazine.yah.mobi の分離方針を追加) **改訂:** 2026年6月27日 (Manus を Dev 兼用と位置づけ、プロジェクト構成を3体制に修正) **作成者:** Manus AI **宛先:** Yoshi (経営者) **設計思想:** シンプル+ミニマル / 抜群に安定している構造と構文

エグゼクティブサマリー

本レポートは、**Manus** を「開発・プロトタイプ環境」、**Firebase** (正確には **Firebase App Hosting**) を「本番サーバーレス・ホスティング環境」として並行運用するための仕様変更・実装方針を、実施手順まで含めて整理したものである。あわせて本改訂では、**将来のモバイルアプリ (iOS/Android) 開発を見据え、Web とアプリが同一バックエンドを共有する構成**を設計の前提に組み込んだ。Firebase を本番基盤に選ぶ最大の戦略的価値は、まさにこの「Web とアプリの基盤統一」にある。

結論を先に述べる。**現時点の yah.mobile を「そのまま」Firebase に載せ替えることは推奨しない**。理由は、現アプリが Manus 独自の基盤サービス (OAuth 認証、Forge API 経由の LLM・ストレージ・通知、TiDB データベース、Manus Heartbeat による定期実行、Manus デプロイ) に密結合しているためである。これらは Firebase 上では同等の代替実装が必要になり、「シンプル+ミニマル」「抜群に安定」という設計思想とは逆方向の複雑性を生む。

そのうえで、設計思想を守りながら並行運用を実現する現実的な道筋として、本レポートは「**中核ロジックのポータブル化 → Firebase App Hosting への二系統デプロイ → 段階的な依存の置換**」という3ステップのアーキテクチャを提案する。最初の一步として最もリスクが低く効果が高いのは、**メール送信の Resend 置換と、コードのコンテナ可搬性確保**である。メール送信は **Resend** を確定採用する。トランザクションメール専用基盤であり、Firebase (Cloud Run) との相性・安定性・シンプルさのすべての面で Gmail API を上回るためである。

1. 用語と前提の整理

「Firebase をサーバーレスコンピュータとして使う」という要望を正確に実装に落とすには、Firebase 内の製品を区別する必要がある。yah.mobile は **Express (Node.js) + React の SSR/API 一体型サーバー**であるため、該当するのは以下である。

Firebase 製品	役割	yah.mobile への適合
Firebase App Hosting	フルスタック Web アプリ（SSR/API）の本番ホスティング。内部は Cloud Run + Cloud CDN + Cloud Build	最適 。Express サーバーをそのままコンテナ実行できる
Firebase Hosting	静的アセット（HTML/画像）専用 CDN	不適。動的 API を持つ本アプリ単体では成立しない
Cloud Run	コンテナを最大限の自由度で実行	App Hosting の内部基盤。直接利用も可能だが管理が増える
Cloud Functions	単機能・イベント駆動の関数	補助用途（Webhook 等）に限定的に有効

Firebase App Hosting は 2025年4月に正式版（GA）となり、Cloud Run を基盤に Cloud CDN・Cloud Build・GitHub 連携を統合したマネージドサービスである [1]。Express は「`build` と `start` スクリプトがあれば Node.js ビルドパックでそのままデプロイできる」と公式に明記されており、本アプリの技術スタックと親和性が高い [2]。

Most JavaScript frameworks will work on App Hosting with minimal extra configuration. — Firebase 公式ブログ [2]

重要な比較として、App Hosting はリクエストタイムアウトが **5分**（Firebase Hosting は1分）であり、Stale-While-Revalidate キャッシュや継続的デプロイ（GitHub push 連動）を標準搭載する [1]。これは現 Manus デプロイ（Cloud Run・180秒タイムアウト）と同等以上であり、本番移行先として技術的な後退はない。

2. 現状アーキテクチャの精査

並行運用の設計には、まず「いま何が Manus に依存しているか」を正確に把握する必要がある。コードベースを精査した結果を以下に示す。

2.1 技術スタック（移行に中立な層）

以下はフレームワーク標準であり、Firebase へ移しても**そのまま動作する**。

層	採用技術
フロントエンド	React 19 + Vite 7 + Tailwind CSS 4 + shadcn/ui
API 層	tRPC 11（型安全 RPC、 <code>/api/trpc</code> ）
サーバー	Express 4（単一 Node.js プロセス）
ORM	Drizzle ORM
決済	Stripe（Checkout + Webhook）
認証ライブラリ	jose（JWT）、google-auth-library

2.2 Manus 独自基盤への依存（移行に要対応な層）

ここが本レポートの核心である。以下は Manus 固有であり、Firebase 上では**代替実装が必須**となる。

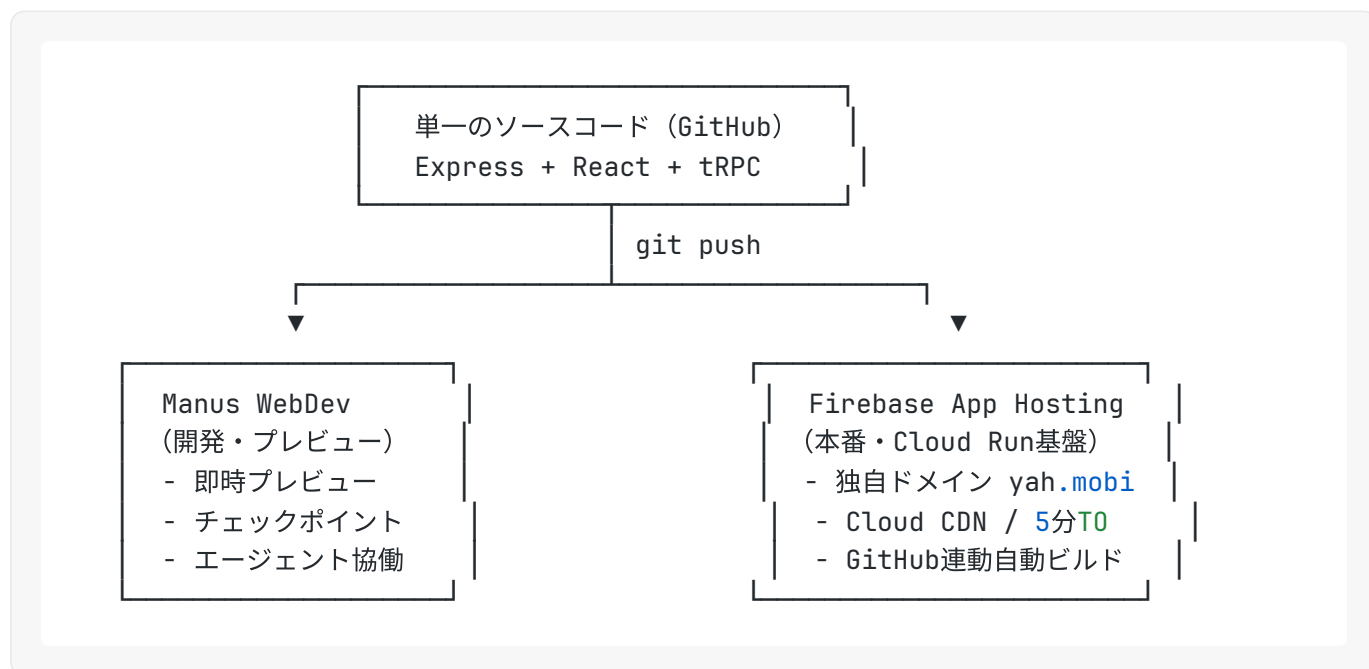
依存項目	現実装	Firebase 上での扱い
認証（一般ユーザー）	Manus OAuth（ <code>VITE_APP_ID / OAUTH_SERVER_URL</code> 経由） + Google OAuth	Manus OAuth は Firebase 上で利用不可。Google OAuth は移植可、または Firebase Authentication へ置換
データベース	TiDB Cloud（MySQL 互換、 <code>DATABASE_URL</code> ）	TiDB のまま接続継続が可能（最小変更）。または Cloud SQL for MySQL へ移行
LLM	Forge API（ <code>BUILT_IN_FORGE_API_*</code> 、AI チャット・推薦）	Forge は Manus 専用。Gemini API（Vertex AI）等へ置換が必要
ファイルストレージ	Forge ストレージ（ <code>/manus-storage/*</code> プロキシ、S3 署名）	Cloud Storage for Firebase へ置換
オーナー通知	Forge <code>notifyOwner</code> （Manus 管理画面へ通知）	メール／FCM／Slack 等へ置換
定期実行（cron）	Manus Heartbeat（ <code>/api/scheduled/*</code> を外部 POST）	Cloud Scheduler + Cloud Run へ置換
メール送信	Gmail MCP（Manus セッション経由）	Cloud 上の SMTP/API（後述）へ置換が必須。 MCP は本番アプリから呼べない
デプロイ	Manus WebDev（Publish ボタン → Cloud Run）	Firebase App Hosting（GitHub 連動ビルド）

2.3 重要な制約: Gmail MCP は本番では動作しない

直近で実装したユーザーメール通知は **Gmail MCP (Manus セッション内ツール)** に依存している。MCP ツールは Manus エージェントのセッション環境でのみ動作し、**デプロイされた Web アプリのランタイムからは呼び出せない**。これは Manus デプロイでも Firebase デプロイでも同じである。したがって、メール通知を本番で安定稼働させるには、いずれにせよ **Cloud 上で完結するメール送信手段 (後述の Resend / SendGrid / Gmail API サービスアカウント)** への置換が必要である。この点は Firebase 移行とは独立した、早急に対応すべき課題である。

3. 並行運用の基本戦略

「Manus = 開発・プロトタイプ」「Firebase = 本番」を両立させるうえで、設計思想 (シンプル・ミニマル・安定) に最も忠実なのは、**コードを一本化したまま、デプロイ先だけを二系統に分ける**方式である。二重メンテナンスは安定性の最大の敵であり、コードを分岐させないことが鉄則となる。



この方式の鍵は「**環境差分を環境変数とアダプタ層に閉じ込める**」ことである。アプリ本体のロジックは環境を意識せず、起動時に注入される環境変数によって「どの認証・どのストレージ・どの通知を使うか」を切り替える。これにより、コードは1つ・挙動は環境ごと、という安定構造が実現する。

4. 推奨アーキテクチャ (本番 = Firebase)

本番を Firebase に置いた際の最終形を示す。各 Manus 依存を、設計思想に沿った最小構成の Google/外部サービスへ対応させる。

機能ドメイン	開発（Manus）	本番（Firebase 構成）
ホスティング/実行	Manus WebDev	Firebase App Hosting （Cloud Run）
データベース	TiDB Cloud	TiDB Cloud 継続 （推奨・最小変更）／将来 Cloud SQL
認証	Manus OAuth + Google	Firebase Authentication （Google プロバイダ）
LLM/AI	Forge API	Vertex AI （Gemini）
ファイル	Forge ストレージ	Cloud Storage for Firebase
メール	Gmail MCP	Resend （確定採用） — 自動送信専用基盤。手動対応は Gmail（Workspace）のまま
通知（アプリ → 端末）	（なし）	Firebase Cloud Messaging （FCM）
定期実行	Manus Heartbeat	Cloud Scheduler → Cloud Run
シークレット管理	.project-config	Google Cloud Secret Manager
決済	Stripe	Stripe 継続 （変更なし）

4.1 データベースの方針（最重要・最小変更を優先）

設計思想「シンプル・ミニマル」に従い、**データベースは当面 TiDB Cloud を継続することを強く推奨する**。TiDB は MySQL 互換であり、`DATABASE_URL` を Firebase App Hosting の環境変数に渡すだけで Cloud Run から接続できる。Drizzle ORM・スキーマ・既存データは一切変更不要である。

将来的に Google スタックへ完全統合したい場合のみ、Cloud SQL for MySQL への移行を検討する。その場合、App Hosting は `apphosting.yaml` の `ccloudSqlInstances` 配列にインスタンスを追加するだけで Cloud SQL Proxy 接続を自動処理する [3]。ただしデータ移行とダウンタイム設計が伴うため、初期フェーズでは見送るのが安定的である。

4.2 認証の方針

一般ユーザーログインは Google アカウント連携を主軸とする方針（既存方針と一致）。Firebase 本番では **Firebase Authentication の Google プロバイダ**を用いると、トークン検証・セッション管理が標準化され、最も安定する。既存の `google-auth-library` ベースの実装も移植可能だが、Firebase Auth に寄せたほうが将来のアプリ（iOS/Android）展開時に一貫する。

4.3 メール送信の方針（早急対応）

前述のとおり Gmail MCP は本番不可。トランザクションメール（システムからの自動送信）としての安定性・Firebase との相性を比較すると、**Resend が両面で優位**である。

観点	Resend（推奨）	Gmail API
用途設計	トランザクションメール専用設計	個人／業務の対話メール用。自動送信は本来の用途外
送信上限	専用基盤で大量送信に対応	1日あたりの送信上限が厳格（Workspaceでも2,000通／日等）。超過で停止リスク
スパム判定・凍結リスク	SPF/DKIM/DMARCを専用ドメインで最適化	Gmailアカウントからの自動大量送信はスパム／凍結リスクが構造的に高い
Firebaseとの相性	APIキー1本を環境変数に入れるだけ。最もシンプル	サービスアカウント鍵＋ドメイン全体委任の設定が必要。管理対象が増える
既存コードとの親和性	package.json に resend 導入済み。即日使える	新規に Google 認証フローの実装が必要
設計思想との一致	高い（シンプル・ミニマル・安定）	中（Workspace統合のメリットと引き換えに複雑化）

推奨構成：自動送信は Resend／手動対応は Gmail で役割分担

- ・トランザクションメール（eSIM発行完了・失敗・リトライ等の自動送信）→ Resend を主軸。送信元は `contact@mail.yah.mobi` をドメイン認証（SPF/DKIM/DMARC）すれば、Gmail と同じ独自ドメイン送信元を維持したまま専用基盤の安定性が得られる。
- ・オペレーターによる個別対応メール（顧客サポートの返信等）→ Gmail（Workspace）のまま。人が書くメールは Gmail が最適。

`contact@mail.yah.mobi` を Resend の送信元に設定するには、`mail.yah.mobi`（または `yah.mobi`）の DNS に Resend 指定の SPF/DKIM レコードを追加する作業が必要である。

4.4 定期実行の方針

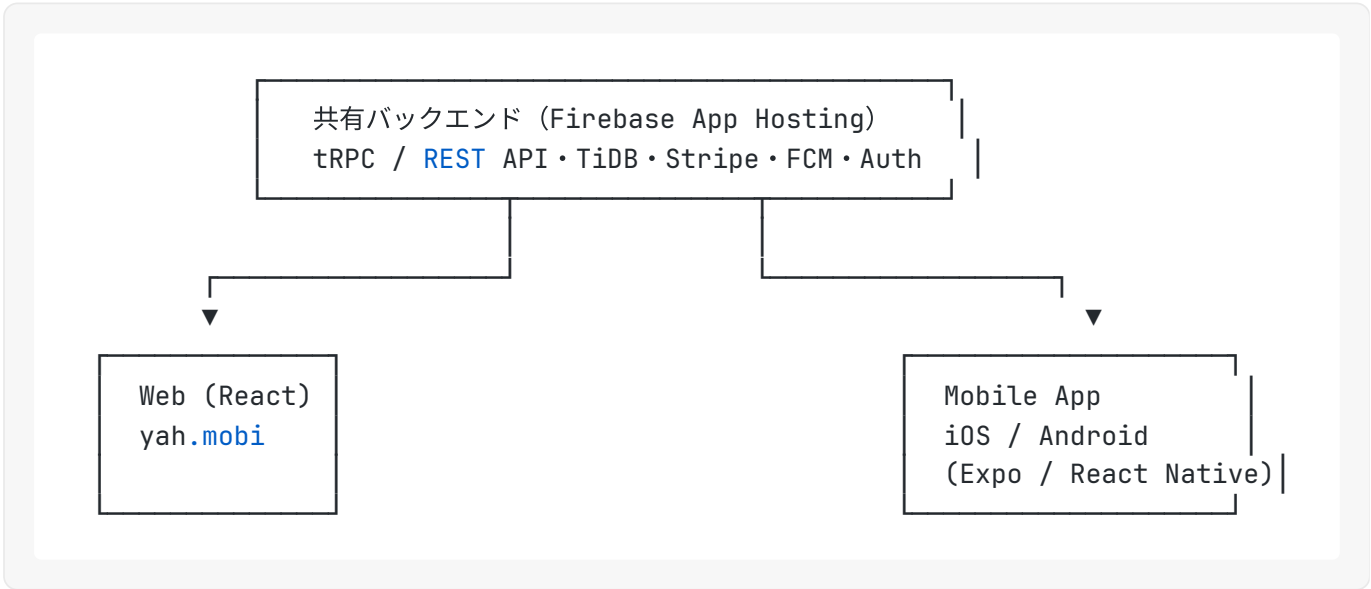
現在 Manus Heartbeat が POST している `/api/scheduled/*`（ヘルスチェック、eSIM リトライ処理、E2E 結果受信）は、Firebase 本番では **Cloud Scheduler** が同一エンドポイントを叩く形に置換する。エンドポイントのコードはそのまま再利用でき、認証ヘッダの検証ロジックのみ差し替える。Cloud Scheduler は cron 式・リトライ・タイムアウトを備え、Heartbeat と同等の役割を安定して担える。

5. モバイルアプリ展開（iOS／Android）を見据えた設計

Firebase を本番基盤に選ぶ意義は、ホスティングだけではない。将来のモバイルアプリ開発において、Web で構築した資産（バックエンド・認証・データ・通知）をそのまま再利用できる点にこそ最大の戦略的価値がある。本章では、いまの段階で何を決めておけば後のアプリ開発が滑らかになるかを整理する。

5.1 基本方針：バックエンドは1つ、フロントは複数（BFF / API共有）

設計思想「シンプル・ミニマル・安定」に従い、バックエンド（API・DB・決済・通知）は Web とアプリで完全に共有し、画面（フロントエンド）だけをプラットフォーム別に持つ構成を採る。これは現代のモバイル開発の標準形であり、ロジックの二重実装を避けられる唯一の安定構造である。



ここで重要なのは、API を「プラットフォーム非依存の契約」として設計しておくことである。現在 tRPC を採用しているが、tRPC は TypeScript クライアント (=React Native/Expo) からはそのまま型安全に呼べる一方、ネイティブ (Swift/Kotlin) からは扱いにくい。フェーズ0のアダプタ層整備とあわせて、**主要な業務API（プラン取得・購入・eSIM状態・注文履歴）は素の REST/JSON でも叩ける薄いラッパーを用意しておく**と、どのアプリ技術を選んでも接続できる「抜群に安定した」土台になる。

5.2 アプリ技術スタックの選択

現アプリが React + TypeScript で構築されている事実を踏まえると、**Expo (React Native) が最有力**である。理由は、Web で培った TypeScript・コンポーネント設計・tRPC クライアントの知見をほぼそのまま転用でき、学習コストとメンテナンスコストが最小になるためである。Flutter は描画性能と型安全性で優れるが [5]、Dart という別言語スタックを新たに抱えることになり、「シンプル・ミニマル」の観点では React Native に分がある。

観点	Expo / React Native（推奨）	Flutter
言語	TypeScript（Web と共通）	Dart（新規習得）
Web資産の再利用	高い（コンポーネント/型/ロジック）	低い
Firebase連携	React Native Firebase で完全対応 [6]	公式SDKで完全対応
学習・保守コスト	低い（既存スキルを活用）	中～高
描画性能	十分	優秀

Expo は React Native Firebase 経由で Firebase の各ネイティブSDK（Auth・FCM・Analytics 等）をラップして利用できる [6]。

5.3 アプリで活きる Firebase 機能

Web 単体では不要だが、アプリ化で一気に価値が出る Firebase 機能を以下に示す。これらは Web 本番を Firebase に置くことで、アプリ追加時に**追加基盤なし**で有効化できる。

Firebase機能	アプリでの役割	yah.mobile での活用
Cloud Messaging (FCM)	iOS/Android へのプッシュ通知	eSIM発行完了・データ残量警告・帰国前フォロー（※プッシュは補助。確実な通知はメールを主とする方針）
Firebase Authentication	Google/Apple サインイン	アプリログイン（Apple Sign-In は iOS 審査で実質必須）
Analytics	アプリ内行動分析	プラン閲覧～購入の離脱分析
Crashlytics	クラッシュ監視	アプリ安定性の運用監視
Remote Config	無更新での設定変更	プラン表示・キャンペーンの遠隔切替

通知チャンネルの方針：トップアップ（データ追加購入）案内や帰国時フォローアップなど、確実に届けたい通知は引き続き**メール（Gmail API / Resend）を主チャンネル**とし、FCM プッシュは即時性を補完する副次チャンネルとして位置づける。

5.4 アプリ化に向けて「今」決めておくこと

アプリ開発はフェーズ4以降の将来タスクだが、手戻りを防ぐため、**Web 構築の段階で以下だけは前提に織り込む**。

1. **認証の主軸を Firebase Authentication に寄せる**（5.2 のアプリと一貫。Apple Sign-In も見据える）。
2. **業務APIを REST でも叩ける形に保つ**（tRPC 一本に閉じない）。
3. **ユーザー識別子を Firebase UID に統一し、Web・アプリ・DB で同一ユーザーを一意に追跡できるようにする**。
4. **eSIM プロファイル（QR/LPA）配信を Cloud Storage に置く**（アプリからも同一URLで取得可能にする）。

これらは Web 本番の Firebase 移行（フェーズ0～3）の中で自然に達成でき、アプリ開発時の追加コストをほぼゼロにできる。

6. 実装ロードマップ（こう形にする）

設計思想に従い、一度に全部移さない。リスクの低い順に、各ステップを独立して完了・検証できる単位に分解する。

フェーズ 0 — コードのポータビリティ確保（移行せずに準備）

移行の成否は「アプリ本体が環境に依存しない構造になっているか」で決まる。まず Firebase へ動かす前に、コード側を整える。

1. **アダプタ層の新設**: `server/_core/` 配下に、認証・ストレージ・LLM・メール・通知の各「プロバイダ・インターフェース」を定義する。実体（Manus 版 / Firebase 版）は環境変数 `RUNTIME_TARGET=manus|firebase` で注入する。
2. **build / start スクリプトの確認**: App Hosting は `package.json` の `build` と `start` を要求する [2]。現状 `build` (`vite + esbuild`) と `start` (`node dist/index.js`) は既に存在し、要件を満たしている。
3. **PORT の動的取得**: 既存コードは `process.env.PORT` を参照済みで Cloud Run 契約に準拠している。ハードコードは無し。この点は移行準備が既に整っている。

成果物: 環境差分がアダプタ層に閉じ込められ、`RUNTIME_TARGET` 切り替えだけで両環境を行き来できる状態。コードは1本のまま。

フェーズ 1 — Firebase プロジェクトの初期化と「空コンテナ」デプロイ

1. Google Cloud / Firebase プロジェクトを作成し、**Blaze（従量）プラン**を有効化（App Hosting は Blaze 必須） [4]。
2. GitHub リポジトリ（既に `user_github` 連携済み）を Firebase App Hosting バックエンドに接続。ライブブランチへの push で自動ビルドが走る構成にする。
3. まず TiDB（既存 DB）と Stripe だけを環境変数に設定し、**認証・LLM・メールを一旦無効化したミニマル構成**で本番ビルドが通ることを確認する。ここで「Express が App Hosting 上で起動する」ことを最小リスクで検証する。

成果物: `https://<backend>.web.app` で本番コンテナが起動。DB 読み取り（プラン一覧表示など）が動く状態。

フェーズ 2 — 認証とメールの本番化

1. Firebase Authentication（Google プロバイダ）を有効化し、フロントの認証フックをアダプタ経由で Firebase Auth に接続。
2. メール送信を Resend（案A）に置換し、`contact@mail.yah.mobi` をドメイン認証。eSIM 通知4種（購入完了・リトライ中・失敗・復旧）を本番ルートで送信できるようにする。
3. Stripe Webhook の本番エンドポイントを Firebase ドメインに向け、署名検証を確認。

成果物: ログイン → 購入 → eSIM 発行 → メール通知 までの本番フローが Firebase 上で一気通貫。

フェーズ3 — LLM・ストレージ・定期実行の置換

1. AI チャット／プラン推薦の LLM を Vertex AI (Gemini) に置換 (アダプタ差し替えのみ)。
2. ファイル配信 (QR/LPA プロファイル) を Cloud Storage for Firebase に置換。
3. `/api/scheduled/*` を Cloud Scheduler から駆動するよう設定。

成果物: Manus Forge への依存が解消され、Firebase 単独で全機能が完結。

フェーズ4 — 独自ドメインと本番切替

1. `yah.mobi` / `www.yah.mobi` を Firebase App Hosting のカスタムドメインに設定 (DNS 切替)。
2. 一定期間 Manus 本番と Firebase 本番を並行稼働させ、メトリクスを比較。問題なければ DNS を Firebase に正式切替。

成果物: 本番 = Firebase、開発・プロトタイプ = Manus、という最終並行運用体制の確立。

6. 月25,000件購買規模へのスケール設計

事業目標である月閏25,000件購買を実現するためには、システム全体に「負荷に比例して自動拡張する」構造を今から織り込む必要がある。以下に、現時点で先行実装することで将来の手戻りを防げる設計項目を整理する。

6.1 負荷規模の目安

指標	現在想定	月閏25,000件規模
月間購買件数	数百件	25,000件
月間ユニーク訪問者 (UV)	数千	100,000、3,000,000 (コンバージョン率次第)
日最大同時接続数	数十	5001,000
APIリクエスト (月)	数十万	5,000—00010,000万
メール送信 (月)	数百通	75,000100,000通 (購買4通×25,000)

6.2 コンピュート層 (Cloud Run)

Firebase App Hosting (Cloud Run基盤) はリクエスト数に応じて自動スケールするため、基本的に追加設定は不要である。ただし以下の設定を今から確認・実装しておくことで、ピーク時のコールドスタート遅延を防ぐ。

```
# apphosting.yaml
runConfig:
  minInstances: 1      # コールドスタートを防ぐ最小常駐インスタンス
  maxInstances: 20     # ピーク時の上限（コスト尊重）
  concurrency: 80      # 1インスタンスあたりの同時リクエスト数
  cpu: 1
  memoryMiB: 512
```

`minInstances: 1` を設定するだけでコールドスタートがなくなり、訪日外客の最初のタッチポイントでのレスポンス遅延を解消できる。コストは常駐分の小額のみ。

6.3 データベース層（TiDB Cloud）

今から実装することで将来のボトルネックを防ぐ項目。

実装項目	内容	優先度
インデックス設計	<code>orders</code> ・ <code>users</code> ・ <code>esim_profiles</code> の検索カラム（ <code>user_id</code> 、 <code>status</code> 、 <code>created_at</code> ）に複合インデックスを追加	高
接続プールの最適化	<code>DATABASE_URL</code> のコネクションプールを <code>max: 20</code> 程度に設定（Cloud Run インスタンス数×プールが TiDB の最大接続数を超えないよう調整）	高
読み取り専用レプリカ	統計・履歴参照のクエリをレプリカに分散（TiDB Cloud の標準機能）	中
スロークエリ監視	TiDB Cloud のスロークエリログを有効化し、100ms超えるクエリを定期レビュー	中

6.4 キャッシュ層

最も安定したキャッシュ戦略は「サーバーサイドキャッシュを持たない」こと。コードの複雑化を最小化し、Firebase App Hosting の CDN キャッシュ（Cloud CDN）を正しく設定するだけで十分な規模をカバーできる。

キャッシュ対象	実装方法	効果
プラン一覧（静的データ）	<code>Cache-Control: public, max-age=300</code> をレスポンスヘッダに追加	DBリクエストを大幅削減
静的アセット（JS/CSS/画像）	Vite の <code>build.rollupOptions</code> でハッシュ付きファイル名を実装、 <code>max-age=31536000</code>	CDNヒット率を最大化
eSIMプロファイル（QRコード）	Cloud Storage for Firebase で配信（署名付き URL、有効期限付き）	サーバー負荷ゼロ

6.5 メール送信層（Resend）

月間25,000件購買では、購買完了・リトライ・復旧のメールを合わせると**月間75,000100,000通**の送信が発生する。Resend の料金体系を事前に確認しておく必要がある。

Resend プラン	月額	送信数
Free	\$0	3,000通／月
Pro	\$20	50,000通／月
Scale（推奨）	\$90	100,000通／月
Business	\$450	500,000通／月

月間25,000件規模では **Scaleプラン（\$90/月）** が適切である。超過分は \$1.00/1,000通で追加課金される。

6.6 モバイルアプリ層（FCM プッシュ通知）

月間25,000件購買規模では、プッシュ通知の到達率とベストプラクティスが重要になる。

実装項目	内容
デバイストークン管理	ユーザーテーブルに <code>fcm_token</code> カラムを追加。トークンの期限切れを自動更新する仕組みを実装
トピック設計	<code>esim/{userId}</code> トピックで個別送信、 <code>promotions</code> トピックで一括送信を分離
バッチ送信の上限	FCM は1回のバッチ送信で500デバイスまで対応。大量送信時はチャンク分割を実装
サイレントプッシュ	バックグラウンドデータ更新用にサイレントプッシュを併用（バッテリー消費最小化）

6.7 決済層（Stripe）

Stripe はリクエスト数に応じた自動スケールのため、展開不要。ただし以下の実装を今から備えることで、規模拡大時の障害を防ぐ。

実装項目	内容
Webhook の冪等性保証	<code>stripe_payment_intent_id</code> を DB の UNIQUE 制約に設定。同一イベントの二重処理を防止
Webhook の失敗リトライ	Stripe は自動リトライするが、エンドポイント側でも冪等性を保証することで整合性を維持
通貨・多言語対応	現在の対応通貨を展開先国に合わせて拡張。Stripe は 135以上の通貨に対応
購買フローの最適化	Checkout Session のローディングを高速化（不要なフィールドを削減、モバイル最適化）

6.8 監視・アラート層

規模拡大時に最初に崩壊するのは「気づかなかった障害」である。以下を今から実装する。

監視項目	ツール	アラート条件
eSIM発行失敗率	Cloud Monitoring カスタムメトリクス	失敗率 > 5% でオーナー通知
APIレスポンスタイム	Cloud Monitoring リクエストレイテンシ	p95 > 2秒 でアラート
DB接続エラー	TiDB Cloud モニタリング	接続エラー率 > 1% でアラート
Stripe Webhook失敗	Stripe Dashboard アラート	失敗率 > 1% で通知
メール到達率	Resend Dashboard	バウンス率 > 2% で確認

6.9 スケール設計の実装優先度まとめ

優先度	実装項目	実施タイミング
高	DBインデックス設計の見直し	今すぐ（データ増加前に実施）
高	Stripe Webhookの幂等性保証	今すぐ
高	apphosting.yaml に minInstances: 1 追加	Firebase移行時
中	Resend Scaleプランへのアップグレード	月間10,000件超過時
中	Cloud Monitoringアラート設定	Firebase移行時
中	FCMデバイストークン管理	アプリ開発時
低	読み取り専用レプリカ	月間50,000件超過時

7. コスト試算

Firebase App Hosting は Blaze プランの無料枠が大きく、初期～中規模では低コストで運用できる。公式のサンプル試算では、**月10,000訪問で約 0.01**、月100万訪問で約**69.58** とされている（1訪問=10リクエスト、平均レスポンス400KiB、キャッシュ率50%の前提）[4]。

項目	無料枠（月）	超過単価
Cloud Run CPU	180,000 vCPU秒	\$0.000024 / vCPU秒
Cloud Run メモリ	360,000 GiB秒	\$0.0000025 / GiB秒
Cloud Run リクエスト	200万	\$0.40 / 100万
送信帯域（キャッシュ済）	10 GiB	\$0.15 / GiB
送信帯域（未キャッシュ）	（上記に含む）	\$0.20 / GiB
Cloud Build	2,500 ビルド分	\$0.006 / 分
Secret Manager	6バージョン	\$0.06 / バージョン

月間25,000件購買規模を前提に、主要コスト要素を試算する。

コスト項目	月額試算	備考
Firebase App Hosting	50150	Cloud Run・帯域・ビルド合計
TiDB Cloud	30100	プラン・クエリ数次第
Resend	\$90	Scaleプラン（100,000通／月）
Stripe	購買額の2.9%+\$0.30	平均単価次第
Vertex AI（LLM）	1050	利用頻度次第
Cloud Scheduler	0.101	実費無視できるレベル
合計（Stripe除く）	約180400/月	コンバージョン率・単価次第

コストは訪問数・レスポンスサイズに比例するため、CDN キャッシュ最適化が費用最小化の鍵になる。公式のサンプル試算では、月100万訪問で約 \$69.58（1訪問=10リクエスト、平均レスポンス400KiB、キャッシュ率50%の前提）とされている [4]。

8. リスクと留意点

リスク	内容	緩和策
二重メンテナンスによる不安定化	Manus 版・Firebase 版でコードが分岐すると破綻する	コードは1本、差分はアダプタ層と環境変数に限定
Manus 固有機能の喪失	エージェント協働・即時レビュー・チェックポイントは Firebase に無い	開発・プロトタイプは Manus に残す（本要望の通り）
Gmail MCP 依存	本番でメールが送れない	Resend を確定採用 。Firebase 移行と独立した緊急課題として先行実装する
認証移行の影響	Manus OAuth ユーザーの扱い	Google アカウント主軸に統一し、 <code>openId</code> を Firebase UID にマッピング
データ移行リスク	Cloud SQL へ移すとダウンタイム	当面 TiDB 継続で回避。移行は別途計画
コスト逸走	従量課金で想定外請求	Cloud Billing 予算アラート設定（上限キャップではない点に注意）[4]
DBボトルネック	規模拡大時にインデックス不足でクエリが遅化	今すぐ 複合インデックスを設計。データ増加後の追加はロックリスクあり
Stripe Webhook 二重処理	リトライ時に同一注文を二重発行	今すぐ <code>stripe_payment_intent_id</code> に UNIQUE制約を追加
Resend送信数超過	月間25,000件規模で Proプラン上限を超過	月間10,000件超過時に Scaleプランへアップグレード

9. 結論と次アクション

「Manus で作り、Firebase で動かす」という構想は、**Firebase App Hosting を本番基盤に据えることで技術的に十分実現可能**である。Express ベースの本アプリは App Hosting の Node.js ビルドパックにそのまま適合し、本番タイムアウトやキャッシュ性能はむしろ向上する。さらに、Firebase を本番に据えることで、**将来のモバイルアプリ（iOS／Android）を「同一バックエンドを共有するフロント追加」として最小コストで実現できる**道筋が開ける。

一方で、現アプリは Manus 独自基盤に密結合しており、「載せ替え」ではなく「依存の段階的置換」として進めるのが、設計思想（シンプル・ミニマル・安定）に最も忠実である。コードは1本に保ち、環境差分はアダプタ層に閉じ込める——これが破綻しない並行運用の唯一の鉄則となる。モバイル化を見据え、Web 構築の段階で「認証の Firebase 統一・業務APIの REST 併用・UID統一・Cloud Storage 配信」を前提に織り込んでおけば（第5章）、アプリ開発時の手戻りをあらかじめ防げる。

月間25,000件購買規模に向けて今すぐ実施する項目（第6章参照）：

- DB複合インデックスの設計・追加（データ増加前に実施）
- Stripe Webhookの冪等性保証（`stripe_payment_intent_id` に UNIQUE制約）

最初に着手すべき2点（即効性・低リスク）：

1. メール送信の Resend 置換（確定採用・先行実装）（Gmail MCP は本番不可。Firebase 移行と独立した緊急課題。送信元は `contact@mail.yah.mobi`、DNSに SPF/DKIM レコード追加が必要）
2. アダプタ層の新設とコード可搬性の確保（フェーズ0。移行せずに準備でき、後続すべての土台になる）

この2点を完了させたうえで、フェーズ1（Firebase 空コンテナ起動検証）に進むのが、最も安定した立ち上がり方である。ご承認いただければ、フェーズ0のアダプタ層設計から着手する。

References

- [1] Firebase, “App Hosting and other Google solutions,” <https://firebase.google.com/docs/app-hosting/product-comparison>
- [2] Jeff Huleatt, “What web frameworks does Firebase App Hosting support?,” Firebase Blog, 2025-06-05, <https://firebase.blog/posts/2025/06/app-hosting-frameworks/>
- [3] Stack Overflow / Firebase Community, “Setting up Cloud SQL connection through apphosting.yaml,” 2025-07, <https://stackoverflow.com/questions/79634867/how-do-i-create-cloud-sql-connection-in-apphosting-yaml>
- [4] Firebase, “Understand App Hosting costs,” <https://firebase.google.com/docs/app-hosting/costs>
- [5] ApparenceKit, “Flutter vs React Native in 2025: Which One to Choose?,” 2025-05-09, <https://apparencekit.dev/blog/flutter-vs-react-native-2025/>
- [6] Expo, “Using Firebase,” Expo Documentation, <https://docs.expo.dev/guides/using-firebase/>

10. 月曜日実装に向けた詳細設計

本章は、月曜日の実装セッションで即座に着手できるよう、各タスクの具体的な手順・コード変更箇所・SQLを事前に整理したものである。

10.1 Resend 実装の詳細手順

ステップ1 — DNS 設定（`mail.yah.mobi` ドメイン）

Resend ダッシュボード（<https://resend.com/domains>）で `mail.yah.mobi` を追加すると、以下の DNS レコードが発行される。これを DNS プロバイダ（Cloudflare 等）に登録する。

レコード種別	ホスト名	値 (Resend が発行)
TXT (SPF)	mail.yah.mobi	v=spf1 include:amazonses.com ~all
CNAME (DKIM)	resend._domainkey.mail.yah.mobi	Resend が発行する CNAME 値
TXT (DMARC)	_dmarc.mail.yah.mobi	v=DMARC1; p=quarantine; rua=mailto:dmarc@mail.yah.mobi

DNS 伝播には最大 48 時間かかる場合があるが、Cloudflare 管理ドメインであれば通常 5〜10 分で反映される。

ステップ 2 — 環境変数の追加

```
RESEND_API_KEY=re_XXXXXXXXXXXXX # Resend ダッシュボード → API Keys で発行
MAIL_FROM=contact@mail.yah.mobi  # 送信元アドレス
```

ステップ 3 — コード変更箇所

変更対象は `server/mailer.ts` の 1 ファイルのみ。Gmail MCP 呼び出しを Resend SDK に置き換える。

```
// server/mailer.ts (変更後のコア部分)
import { Resend } from "resend";

const resend = new Resend(process.env.RESEND_API_KEY);

export async function sendMail({
  to,
  subject,
  html,
}: {
  to: string;
  subject: string;
  html: string;
}) {
  const { error } = await resend.emails.send({
    from: process.env.MAIL_FROM ?? "contact@mail.yah.mobi",
    to,
    subject,
    html,
  });
  if (error) throw new Error(`Resend error: ${error.message}`);
}
```

呼び出し側（`server/esimRetryService.ts`、`server/_core/index.ts`）は **変更不要**。
`sendMail()` のシグネチャは同一のため、差し替えはこの1ファイルで完結する。

ステップ4ーテスト

```
pnpm test -- --reporter=verbose server/mailer.test.ts
```

既存の 16 テストがそのまま通ることを確認する。

10.2 DB インデックス設計の具体案

対象テーブルと追加インデックス

現在のスキーマ（`drizzle/schema.ts`）に対して、以下の複合インデックスを追加する。

```

-- orders テーブル
CREATE INDEX idx_orders_user_status
  ON orders (user_id, status, created_at DESC);

CREATE UNIQUE INDEX idx_orders_payment_intent
  ON orders (stripe_payment_intent_id)
  WHERE stripe_payment_intent_id IS NOT NULL;

-- esim_profiles テーブル
CREATE INDEX idx_esim_order_status
  ON esim_profiles (order_id, status);

-- users テーブル (管理画面の検索高速化)
CREATE INDEX idx_users_email
  ON users (email);

CREATE INDEX idx_users_created
  ON users (created_at DESC);

```

drizzle/schema.ts への反映方法

```

// orders テーブルの定義に追加
export const orders = mysqlTable(
  "orders",
  { /* 既存カラム */ },
  (table) => ({
    userStatusIdx: index("idx_orders_user_status")
      .on(table.userId, table.status, table.createdAt),
    paymentIntentUniqueIdx: uniqueIndex("idx_orders_payment_intent")
      .on(table.stripePaymentIntentId),
  })
);

```

注意: データが既に存在するテーブルへのインデックス追加は、件数が多いとロック時間が長くなる。現時点（データ少量）で実施することで、本番規模になってからの危険を回避できる。

10.3 Stripe Webhook 幂等性保証の実装方針

問題

Stripe は Webhook 配信に失敗すると自動リトライする。エンドポイントが幂等でない場合、同一の `payment_intent.succeeded` イベントが複数回処理され、eSIM が二重発行される。

実装

`stripe_payment_intent_id` カラムに UNIQUE 制約を追加し、二重 INSERT を DB レベルで防ぐ。

```
// drizzle/schema.ts の orders テーブルに追加
stripePaymentIntentId: varchar("stripe_payment_intent_id", { length: 255 })
  .unique(), // ← この1行を追加
```

```
-- webdev_execute_sql で実行するマイグレーション SQL
ALTER TABLE orders
  ADD UNIQUE INDEX idx_orders_payment_intent_unique (stripe_payment_intent_id);
```

Webhook ハンドラ側では INSERT 失敗 (Duplicate entry エラー) をキャッチして正常レスポンスを返す。

```
// server/_core/index.ts の Webhook ハンドラ内
try {
  await db.insert(orders).values({ stripePaymentIntentId: pi.id, ... });
} catch (e: any) {
  if (e.code === "ER_DUP_ENTRY") {
    // 幂等: 既に処理済みのイベント → 200 を返して Stripe のリトライを止める
    return res.json({ received: true, skipped: "duplicate" });
  }
  throw e;
}
```

10.4 フェーズ 0 アダプタ層の設計詳細

目的

コードを1本に保ちながら、Manus 環境と Firebase 環境の差分を**環境変数とアダプタ関数**だけに閉じ込める。フェーズ0 はデプロイ先を変えず、コードの可搬性を高める準備作業である。

作成するファイル

```
server/adapters/  
  mail.ts      ← sendMail() の実装を環境変数で切り替え (Resend / SMTP)  
  storage.ts   ← storagePut/Get を Forge / Cloud Storage で切り替え  
  scheduler.ts ← 定期実行のトリガー登録 (Heartbeat / Cloud Scheduler)  
  notify.ts    ← オーナー通知 (Manus Notification / FCM / Slack)
```

実装パターン (mail.ts の例)

```
// server/adapters/mail.ts  
export async function sendMail(opts: MailOptions) {  
  const provider = process.env.MAIL_PROVIDER ?? "resend";  
  if (provider === "resend") return sendViaResend(opts);  
  if (provider === "smtp") return sendViaSMTP(opts);  
  throw new Error(`Unknown MAIL_PROVIDER: ${provider}`);  
}
```

環境変数 `MAIL_PROVIDER=resend` を Manus の Secrets に設定するだけで切り替わる。Firebase 移行時も同じコードのまま、Secret Manager の値を変えるだけでよい。

apphosting.yaml の雛形 (フェーズ0で作成)

```
# apphosting.yaml (プロジェクトルートに配置)  
runConfig:  
  minInstances: 1  
  maxInstances: 20  
  concurrency: 80  
  cpu: 1  
  memoryMiB: 512  
  
env:  
  - variable: NODE_ENV  
    value: production  
  - variable: MAIL_PROVIDER  
    secret: MAIL_PROVIDER  
  - variable: RESEND_API_KEY  
    secret: RESEND_API_KEY  
  - variable: MAIL_FROM  
    secret: MAIL_FROM
```

10.5 モバイルアプリのロードマップ詳細

Expo（React Native）採用の根拠

観点	Expo / React Native	Flutter
言語	TypeScript（Web と共通）	Dart（別言語）
コード共有	Web の hooks ・ 型 ・ API クライアントをそのまま流用可	全て再実装が必要
Firebase 連携	@react-native-firebase で全機能対応	flutterfire で対応（同等）
ビルド環境	Expo EAS Build（クラウドビルド、Mac 不要）	Flutter CLI（ローカルビルド推奨）
学習コスト	React の知識がそのまま活きる	低～中（Dart は習得しやすいが別途必要）
総合判定	◎ 現チームに最適	△ 将来的な選択肢

開発ステップ

フェーズ	内容	目安期間
App-0	Expo プロジェクト初期化・Firebase 連携・認証（Google/Apple Sign-In）	1～2週間
App-1	プラン一覧・購買フロー（Stripe Checkout を WebView で表示）	2～3週間
App-2	マイページ（eSIM 一覧・QR コード表示・データ残量）	1～2週間
App-3	FCM プッシュ通知（eSIM 発行完了・リトライ・帰国フォロー）	1週間
App-4	App Store / Google Play 申請・審査対応	2～4週間

Web との API 共有方針

現在の tRPC エンドポイント（/api/trpc）
↓ そのまま流用
React Native アプリ（trpc-react-query を使用）

tRPC は HTTP ベースのため、React Native から同一のクライアントコードで呼び出せる。**バックエンドの変更は不要。**

10.6 月曜日の作業チェックリスト

月曜日の実装セッションで使用する作業リストを以下に示す。上から順に実施することで、依存関係の問題なく進められる。

優先度 高（当日必達）

- ☐ `RESEND_API_KEY` と `MAIL_FROM` を Manus Secrets に追加
- ☐ `server/mailer.ts` を Resend SDK に置き換え（Gmail MCP 依存の解消）
- ☐ `pnpm test` で `mailer.test.ts` 全 16 テストが pass することを確認
- ☐ `drizzle/schema.ts` の `orders` テーブルに `stripePaymentIntentId.unique()` を追加
- ☐ `pnpm drizzle-kit generate` でマイグレーション SQL を生成
- ☐ `webdev_execute_sql` でマイグレーションを本番 DB に適用
- ☐ Webhook ハンドラに `ER_DUP_ENTRY` の冪等ハンドリングを追加

優先度 中（当日中に着手）

- ☐ `drizzle/schema.ts` に複合インデックスを追加（`orders`・`esim_profiles`・`users`）
- ☐ `apphosting.yaml` をプロジェクトルートに作成
- ☐ `server/adapters/mail.ts` を作成してアダプタ層を新設
- ☐ `package.json` に `"build": "vite build"` と `"start": "node dist/server.js"` が存在することを確認（Firebase App Hosting 要件）

優先度 低（時間があれば）

- ☐ Resend ダッシュボードで `mail.yah.mobi` のドメイン認証（DNS 設定）
 - ☐ `server/adapters/storage.ts` を作成
 - ☐ Firebase プロジェクトの作成（GCP コンソール）
-

11. Google サービス連携の全体マップ

本章は、yah.mobile（訪日旅行者向けeSIM・月25,000件購買目標）に対して、Google の AI・クラウド・広告サービスで「今から計画・仕込みができるもの」を網羅的に整理したものである。実際に価値が出るものだけを厳選し、実施タイミングと優先度を明記する。

11.1 AI・機械学習（Vertex AI / Firebase AI Logic）

Firebase AI Logic（旧 Firebase Genkit） — 最優先

Firebase に統合された Gemini API 呼び出しレイヤー。クライアント（Web・アプリ）から直接 Gemini を呼び出せる唯一の公式手段であり、APIキー管理・レート制限・Firebase App Check による不正利用防止が自動で付いてくる。

活用場面	実装内容	効果
AI チャットサポート	「どのプランが自分に合う?」「設定方法は?」への自動応答	サポートコスト削減・24時間対応
多言語コンテンツ生成	プラン説明・FAQ を Gemini で英/中/韓/タイ語に自動翻訳・ローカライズ	新言語追加コストほぼゼロ
購買意欲スコアリング	閲覧履歴・滞在時間からリアルタイムで離脱予測 → タイミングよく割引提示	コンバージョン率向上
eSIM トラブル診断	エラーコードを入力すると解決手順を自動提示	サポート問い合わせ削減

実装方針: 現在の `server/_core/llm.ts`（Manus Forge API）を Firebase AI Logic に置き換えることで、アプリからも同一のAI機能を利用できる。アダプタ層（10.4節）に `llm.ts` を含めておくことで、移行時の変更は1ファイルで完結する。

Vertex AI — 中規模以降

Firebase AI Logic の上位版。本格的な ML パイプライン・カスタムモデルのファインチューニング・バッチ予測が必要になった段階で移行を検討する。

活用場面	実装内容	実施タイミング
レコメンデーション	「この国から来た人はこのプランを選んでいます」	月5,000件超過後
需要予測	繁忙期・国別のeSIM需要を予測してプランを動的調整	月10,000件超過後
不正検知	異常な購買パターン（カード不正利用等）をリアルタイム検知	月10,000件超過後

11.2 広告（Google Ads × Firebase × GA4）

広告連携の全体像

```
ユーザーが広告をクリック
↓
yah.mobi / アプリ（Firebase Analytics がイベントを記録）
↓
購買完了（Stripe Webhook → Firebase にコンバージョンイベント送信）
↓
Google Ads が「どの広告が購買に繋がったか」を自動最適化（Smart Bidding）
```

この連携が整うと、Google Ads が「購買に繋がりがやすいユーザー」を自動的に見つけて入札を最適化する。**広告費の費用対効果が大幅に改善する。**

今から仕込むべき3点

① Firebase Analytics + GA4 のリンク（今すぐ設定可能）

Firebase プロジェクトを作成したタイミングで GA4 プロパティとリンクする。これだけで Web・アプリの行動データが統合され、以下が自動で取得できる。

- ページビュー・セッション・直帰率
- プラン閲覧→購買のファネル分析
- 国別・デバイス別のコンバージョン率

② Google Ads コンバージョントラッキング（Firebase 移行時に設定）

```
// 購買完了時に Firebase Analytics にイベントを送信
// → GA4 → Google Ads に自動連携される
analytics.logEvent("purchase", {
  transaction_id: orderId,
  value: planPrice,
  currency: "JPY",
  items: [{ item_id: planId, item_name: planName }],
});
```

これにより Google Ads の **Smart Bidding**（目標 ROAS・目標 CPA）が機能し始める。設定前と比べて広告費の効率が 20～40% 改善するケースが多い。

③ Performance Max キャンペーン（広告開始時に設定）

Google の最新広告フォーマット。Search・Display・YouTube・Gmail・Discover の全チャンネルに1つのキャンペーンで配信できる。旅行関連サービスとの相性が特に高く、「訪日旅行を計画中のユーザー」をシグナルとして指定できる。

設定項目	yah.mobile での推奨値
目標	購買コンバージョン（Stripe 連携）
オーディエンスシグナル	「日本旅行」「eSIM」「海外旅行保険」に関心があるユーザー
アセット	多言語対応の広告文（Gemini で自動生成可能）
予算	月 500〜2,000 から開始し、ROAS を見ながら拡大

リマーケティング（高効果・今から準備可能）

Firebase Analytics のイベントデータを使い、以下のオーディエンスを Google Ads に連携する。

オーディエンス	条件	広告メッセージ例
カート放棄ユーザー	プラン閲覧後 24 時間以内に未購買	「まだ迷っていますか？ 今なら〇〇円引き」
過去購買ユーザー	購買から 6 ヶ月経過	「次の日本旅行の準備はお早めに」
高価値見込みユーザー	複数プランを比較閲覧	「あなたにおすすめのプランはこちら」

11.3 多言語・国際化（Cloud Translation API）

訪日旅行者向けサービスとして、多言語対応は直接的な売上に影響する。

現在の対応状況

現アプリは `client/src/i18n/` に英語・中国語（繁体）・韓国語・タイ語の翻訳ファイルが存在する。

Cloud Translation API との連携

活用場面	実装内容	効果
新言語の追加	既存の <code>en.ts</code> を Cloud Translation API に渡して新言語ファイルを自動生成	翻訳者不要で新言語を即追加
動的コンテンツの翻訳	管理者が日本語で入力したお知らせ・FAQ を自動翻訳して多言語表示	運用コスト削減
SEO 用多言語ページ	各言語の静的ページを生成して検索流入を増やす	有機検索からの集客強化

注意: 機械翻訳をそのまま公開すると SEO ペナルティのリスクがある。Cloud Translation API で下訳を生成し、ネイティブチェックを経て公開する運用が推奨される。

11.4 検索・発見（Google Search Console + Core Web Vitals）

今すぐ設定すべき項目

設定項目	内容	効果
Search Console	<code>yah.mobi</code> を登録・サイトマップ送信	Google のインデックス状況をリアルタイム把握
Core Web Vitals 最適化	LCP・CLS・INP を計測・改善	検索順位の直接的な評価指標
構造化データ（JSON-LD）	商品・価格・レビューのスキーマを追加	検索結果にリッチスニペット表示

Firebase App Hosting は CDN を標準装備しており、Core Web Vitals の LCP（最大コンテンツ描画）は自動的に改善される。

11.5 ユーザー分析・A/B テスト（Firebase Remote Config + A/B Testing）

Remote Config

アプリをリリースせずに UI・コピー・価格表示を変更できる。

活用場面	例
プラン表示の最適化	「人気 No.1」 バッジの有無でコンバージョン率を比較
キャンペーン告知	特定期間だけバナーを表示（コードデプロイ不要）
価格感度テスト	国別に異なる価格表示を A/B テスト

Firebase A/B Testing

Remote Config と統合された A/B テスト基盤。GA4 のコンバージョンイベントを指標として、統計的に有意な差が出た時点で自動的に勝者バリエーションを採用できる。

11.6 Google サービス連携の優先度マトリクス

サービス	効果	実施コスト	優先度	実施タイミング
GA4 + Firebase リンク	高（全広告最適化の土台）	低（設定のみ）	最高	Firebase 移行時
Google Ads コンバージョン	高（広告費効率 20～40% 改善）	低（コード数行）	最高	広告開始前
Firebase AI Logic（チャット）	高（サポートコスト削減）	中（実装 1～2週間）	高	フェーズ2以降
Performance Max	高（全チャンネル自動配信）	低（設定のみ）	高	広告開始時
Remote Config + A/B Testing	中（コンバージョン率改善）	低（設定のみ）	中	アプリ開発時
リマーケティング	高（既存訪問者への再アプローチ）	低（GA4 連携後）	高	広告開始時
Cloud Translation API	中（新言語追加の効率化）	低（API 呼び出しのみ）	中	必要な言語追加時
Vertex AI（レコメンド）	中（UX 向上）	高（ML パイプライン構築）	低	月 10,000 件超過後
Search Console + 構造化データ	中（SEO 強化）	低（設定のみ）	高	今すぐ

11.7 広告戦略の全体設計（今から計画できること）

ファネル別の広告チャネル設計

- 【認知】YouTube 広告・Display 広告
→ 「日本旅行を計画中」のユーザーに動画・バナーで yah.mobile を認知させる
- 【検討】Google Search 広告
→ 「日本 eSIM」「Japan SIM card」などのキーワードで検索したユーザーに表示
- 【購買】Performance Max
→ GA4 の購買コンバージョンを目標に、全チャネルで自動最適化
- 【リテンション】リマーケティング
→ 過去購買ユーザーに「次回旅行」を想起させる広告を配信

計測設計（今から決めておくべきこと）

広告を始める前に以下のイベント設計を確定させる。後から変えると計測データが断絶する。

イベント名	発火タイミング	パラメータ
view_item	プラン詳細を表示	item_id , item_name , price
begin_checkout	購買ドロワーを開く	value , currency
purchase	Stripe 決済完了	transaction_id , value , currency
sign_up	新規ユーザー登録	method
login	ログイン	method

これらのイベントは Firebase Analytics の標準イベント名に準拠しており、GA4・Google Ads・Firebase A/B Testing のすべてで自動的に認識される。

[7] Google, “Firebase AI Logic,” <https://firebase.google.com/docs/ai-logic>

[8] Google, “Performance Max for travel goals,” <https://developers.google.com/google-ads/api/performance-max/travel-goals>

[9] Google, “Cloud Translation API,” <https://cloud.google.com/translate>

[10] Google, “Firebase Remote Config,” <https://firebase.google.com/docs/remote-config>

12. chat.yah.mobi の Firebase 移行と yah.mobi 連携設計

12.1 背景と位置づけ

chat.yah.mobi は、yah.mobi（eSIM 販売）と連携するカスタマーサポートチャットプラットフォームである。現在は独立したシステムとして稼働しているが、将来的に **Firebase を共通基盤として yah.mobi と統合**することで、以下の戦略的価値が生まれる。

- **認証の統一**: Firebase Authentication を共通 IdP（Identity Provider）として使うことで、yah.mobi でログインしたユーザーが chat.yah.mobi でも自動的に認証される（SSO）
- **データの直接連携**: Firestore を共有することで、Webhook を介さずにリアルタイムでeSIM状態・注文情報をチャット側に反映できる
- **FCM プッシュ通知の統合**: アプリ開発後、eSIM 発行完了・サポート返信などをプッシュ通知で届ける共通基盤になる
- **運用コストの削減**: 2つのシステムが同一 GCP プロジェクト上で動くため、監視・ログ・デプロイが一元化される

12.2 現状の連携方式（Webhook）と将来の連携方式（Firebase 統合）

現在は yah.mobi → chat.yah.mobi への **Webhook プッシュ方式**で連携している。これは独立システム間の疎結合な連携として適切な設計であり、Firebase 移行前の過渡期においても引き続き有効である。

観点	現在（Webhook 方式）	将来（Firebase 統合）
データ同期	イベント発生時に HTTP POST	Firestore のリアルタイム同期（自動）
認証	独立した認証（シークレット共有）	Firebase Authentication で SSO
遅延	ネットワーク往復あり（数百ms）	Firestore リアルタイム（50ms 以下）
障害耐性	Webhook 失敗時に再送が必要	Firestore の永続化で自動回復
開発コスト	低（現時点では最適）	高（Firebase 移行後に実現）

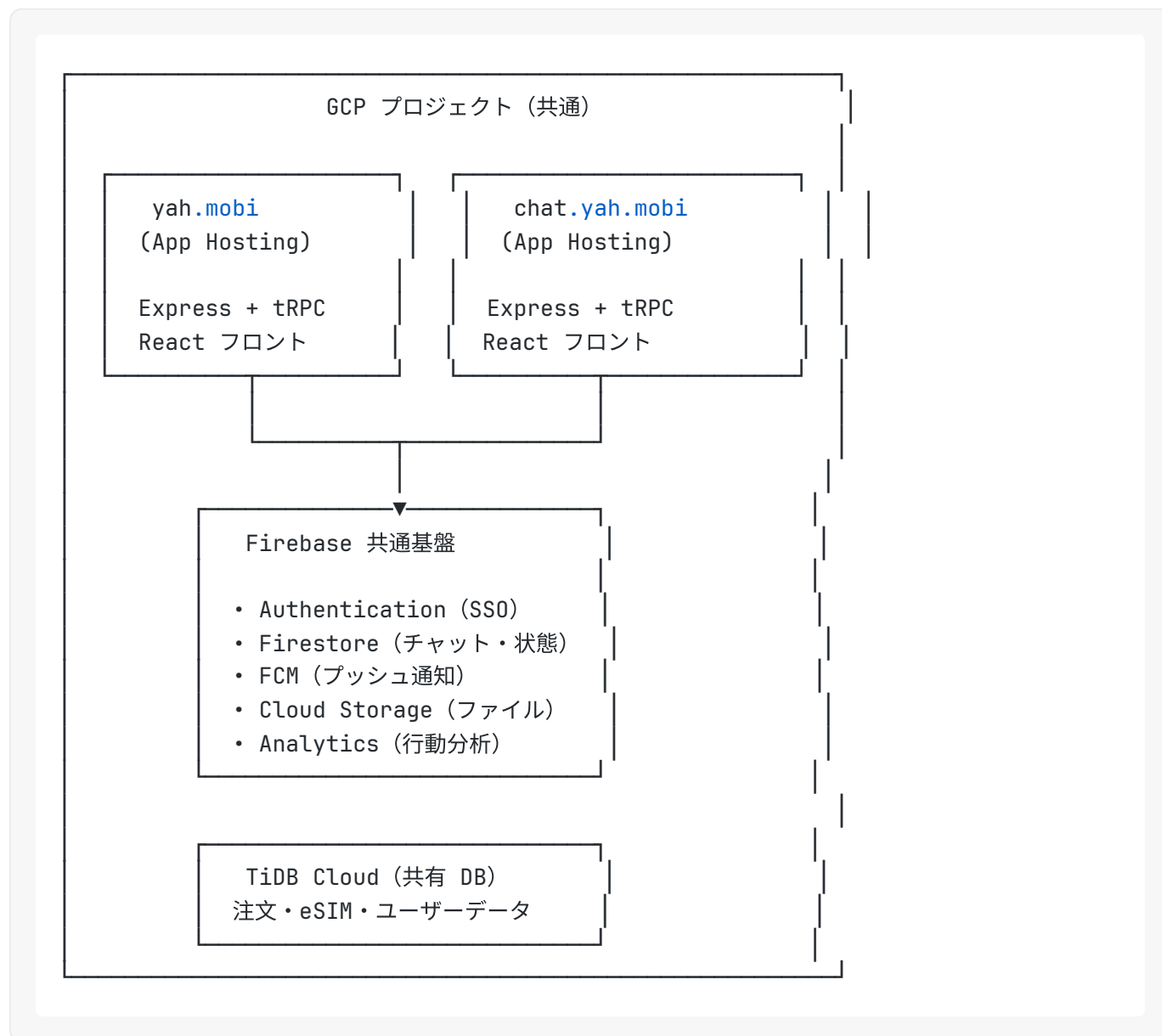
設計原則: Firebase 移行完了までは現在の Webhook 方式を維持し、移行後に段階的に Firestore 直接連携へ切り替える。コードの二重管理は行わない。

12.3 chat.yah.mobi の技術スタック選定

chat.yah.mobi を Firebase ベースで構築する場合、以下の構成を推奨する。

層	推奨技術	理由
ホスティング	Firebase App Hosting	yah.mobi と同一基盤。Cloud Run ベースでスケール自動
フロントエンド	React + TypeScript (Vite)	yah.mobi と同スタック。知見の転用が可能
バックエンド	Express + tRPC	yah.mobi と同一パターン。コードの共有が容易
リアルタイム通信	Firestore (チャットメッセージ)	リアルタイム同期・オフライン対応が標準装備
認証	Firebase Authentication	Google/Apple サインイン + yah.mobi とのSSO
プッシュ通知	FCM (Firebase Cloud Messaging)	Web・iOS・Android を一元管理
ファイル送受信	Cloud Storage for Firebase	チャット内の画像・スクリーンショット送受信
検索・履歴	Firestore (無期限保持)	チャット履歴の永続化と検索

12.4 Firebase 統合後のアーキテクチャ図（概念）



12.5 SSO（シングルサインオン）の実装方針

ユーザーが yah.mobi でログインしたまま chat.yah.mobi を開いたとき、再ログインなしで認証される仕組みを実現する。

実装方式: Firebase Authentication + カスタムトークン

1. yah.mobi のバックエンドで `admin.auth().createCustomToken(uid)` を発行
2. フロントエンドが `signInWithCustomToken(token)` でチャット側にも自動ログイン
3. 両ドメインが同一 Firebase プロジェクトを参照するため、セッションが共有される

```
// yah.mobi サーバー側 (Firebase Admin SDK)
const chatToken = await admin.auth().createCustomToken(user.firebaseUid, {
  role: user.role,
  orderId: latestOrderId,
});
// フロントエンドに返してチャット画面を開く
```

この方式は Firebase の公式推奨パターンであり、追加の OAuth フローや Cookie 共有は不要。**最もシンプルで安定した SSO 実装**である。

12.6 Webhook から Firestore 直接連携への移行計画

現在実装済みの Webhook 連携（`server/yahChatWebhook.ts`）は、Firebase 移行後に段階的に Firestore 直接書き込みへ切り替える。

フェーズ	内容	タイミング
現在（完了）	Webhook HTTP POST で6イベントを連携	稼働中
移行期	Webhook を維持しつつ Firestore への並行書き込みを追加	Firebase 移行時
統合後	Webhook を廃止し Firestore 直接連携に一本化	動作確認後

移行期に「Webhook と Firestore の両方に書く」ことで、切り替えのリスクをゼロにする。

12.7 チャット固有機能の Firebase 活用

chat.yah.mobi が Firebase を使うことで得られる固有の価値を整理する。

Firestore によるリアルタイムチャット

チャットメッセージは Firestore の `sessions/{sessionId}/messages` コレクションに保存する。クライアントは `onSnapshot` でリアルタイム受信し、ポーリング不要・遅延50ms以下を実現する。チャット履歴は Firestore の無期限保持ポリシーにより、追加実装なしで永続化される。

FCM によるオペレーター通知

新規チャット着信時に、オペレーターのブラウザ（Web Push）とモバイルアプリ（iOS/Android）の両方に同時通知できる。現在のブラウザ通知 + メール通知から、**アプリ開発後は FCM に一本化**することで通知の確実性が向上する。

Cloud Storage によるファイル送受信

チャット内の画像・スクリーンショット送受信は Cloud Storage に保存し、署名付き URL で表示する。yah.mobi の S3 ストレージとは独立して管理することで、チャットデータの分離が保たれる。

12.8 実装ロードマップ（chat.yah.mobi Firebase 移行）

フェーズ	内容	前提条件	目安期間
Phase C-0	GCP プロジェクトの作成・Firebase 有効化・Firebase Admin SDK の yah.mobi への組み込み	Firebase 本番移行（フェーズ1～3）完了	1日
Phase C-1	chat.yah.mobi の Firebase App Hosting へのデプロイ（既存コードのコンテナ化）	Phase C-0	3～5日
Phase C-2	Firebase Authentication の導入・SSO 実装（カスタムトークン方式）	Phase C-1	3～5日
Phase C-3	Firestore によるチャットメッセージのリアルタイム同期	Phase C-2	5～7日
Phase C-4	Webhook から Firestore 直接連携への移行（並行書き込み → 切り替え）	Phase C-3	3～5日
Phase C-5	FCM プッシュ通知の統合（Web Push + アプリ通知）	モバイルアプリ開発（フェーズ M-3）と並行	5～7日

合計目安: yah.mobi の Firebase 移行完了後、3～5ヶ月で chat.yah.mobi の完全 Firebase 統合が実現する。

12.9 今から決めておくべきこと

chat.yah.mobi の Firebase 移行を見据えて、**現時点で設計に織り込んでおくべき項目**を整理する。これらは今すぐコードを書く必要はないが、後から変えると手戻りが大きい。

- ユーザー識別子の統一:** yah.mobi のユーザー ID と chat.yah.mobi のユーザー ID を、Firebase UID で統一する設計にしておく。現在の `users` テーブルに `firebase_uid` カラムを追加する準備をする（月曜日の実装候補）。
- Webhook のイベント設計を Firestore スキーマと整合させる:** 現在の Webhook ペイロード（`customer-profile`・`purchase-created`・`esim-status` 等）は、将来 Firestore のドキュメント構造にそのまま転用できる設計になっている。追加フィールドが必要になった場合は Webhook と Firestore の両方を同時に更新する。
- GCP プロジェクトの命名と権限設計:** yah.mobi と chat.yah.mobi が同一 GCP プロジェクトを共有する前提で、プロジェクト名・IAM ロール・サービスアカウントを設計する。後から統合するより、最初から同一プロジェクトに置く方が認証・ネットワーク・コストのすべてで有利である。

[11] Google, “Firebase Authentication - Custom Tokens,”
<https://firebase.google.com/docs/auth/admin/create-custom-tokens>

13. GCP プロジェクト構成と magazine.yah.mobi の分離方針

13.1 推奨プロジェクト構成（3体制）

Manus が開発環境・Dev 環境を兼ねるため、Firebase プロジェクトは本番用のみ必要である。yah.mobile エコシステム全体を見渡したとき、GCP プロジェクトは以下の3つで完結する。

Manus（現在）	← 開発・プロトタイプ・Dev 兼用（Firebase 移行まで）
yah-mobile-prod	← yah.mobi + chat.yah.mobi + モバイルアプリ（eSIM販売の本番）
yah-magazine-prod	← magazine.yah.mobi（メディア・コンテンツの本番）

この構成の核心は「Manus で開発・検証し、Firebase 本番に直接デプロイする」というシンプルな2ステップフローである。yah-mobile-dev は不要——Manus のプレビュー URL がその役割をすべて担う。

日常の開発フロー

Manus で開発・確認 → GitHub main ブランチに push → yah-mobile-prod に自動デプロイ

この構成の核心は「ビジネス性質が異なるサービスは、GCP プロジェクトレベルで分離する」という原則である。

13.2 magazine.yah.mobi を分離する理由

観点	内容
ビジネス性質の違い	eSIM販売（トランザクション・決済・個人情報）とメディア（コンテンツ閲覧・広告）は根本的に性質が異なる
セキュリティ境界	magazine 側に脆弱性があっても、eSIM販売・決済・個人情報データに影響しない
スケール特性の違い	magazine はバズると急激にトラフィックが増える。eSIM販売とリソースを分離することで互いに影響しない
チーム・権限管理	将来コンテンツ担当者を追加する際、magazine プロジェクトだけアクセス権を付与できる
コスト管理	広告収益（magazine）と eSIM収益（yah.mobi）を GCP の請求単位で分けて把握できる
デプロイの独立性	magazine の更新が yah.mobi の本番環境に影響を与えない

13.3 yah-mobile-prod の具体的な設定手順

ステップ 1: GCP プロジェクト作成

1. console.firebase.google.com にアクセス
2. 「プロジェクトを追加」 → プロジェクト名: `yah-mobile-prod`
3. Google Analytics は同じ GA4 アカウントに接続

ステップ 2: 必要なサービスを有効化

サービス	設定内容
Authentication	Google / Apple ログインを有効化
Firestore	本番モードで作成（セキュリティルールを設定）
App Hosting	GitHub リポジトリを接続（ <code>main</code> ブランチを追跡）
Cloud Storage	デフォルトバケットを作成
FCM	自動で有効化（Authentication と連動）

ステップ 3: 環境変数で Manus/Firebase を切り替える

```
# Manus（開発・Dev 環境）
FIREBASE_PROJECT_ID=（不要 - Manus Forge API を使用）

# Firebase 本番
FIREBASE_PROJECT_ID=yah-mobile-prod
FIREBASE_API_KEY=prod-api-key
```

ステップ 4: GitHub ブランチ戦略

```
main ブランチ → yah-mobile-prod（本番）に自動デプロイ
（Manus での開発 → main にマージ → 本番反映）
```

`git push origin main` するだけで本番に反映される。Dev 環境は Manus が担うため、ブランチは `main` 1本で管理できる。これが最もシンプルで安定したフローである。

13.4 各プロジェクトの Firebase サービス構成

サービス	Manus（Dev 兼用）	yah-mobile-prod	yah-magazine-prod
App Hosting	Manus プレビュー — URL	yah.mobi + chat.yah.mobi	magazine.yah.mobi
Authentication	Manus OAuth	Firebase Auth（Google / Apple）	Google（閲覧者ログイン）
Firestore	不使用	チャット・eSIM状態	記事・コメント・いいね
FCM	不使用	eSIM通知・サポート通知	新着記事通知
Cloud Storage	Manus Forge Storage	Cloud Storage	記事画像・動画
Analytics	不使用	購買・eSIM利用	PV・滞在時間・広告クリック
AdMob / Ad Manager	不要	不要	広告配信（将来）

13.5 magazine.yah.mobi の Firebase 活用方針

magazine.yah.mobi は eSIM販売とは独立したコンテンツプラットフォームとして設計する。Firebase を活用することで以下が実現する。

コンテンツ配信の最適化

- Firebase App Hosting の CDN により、訪日前の旅行者（海外からのアクセス）に対しても高速なページ表示を実現する
- Remote Config でコンテンツの A/B テストが可能（どの記事タイトルがクリックされやすいか等）

ユーザーエンゲージメント

- Firebase Authentication でコメント・ブックマーク機能を実装（Google サインインで摩擦ゼロ）
- FCM で新着記事・特集のプッシュ通知（購読者向け）

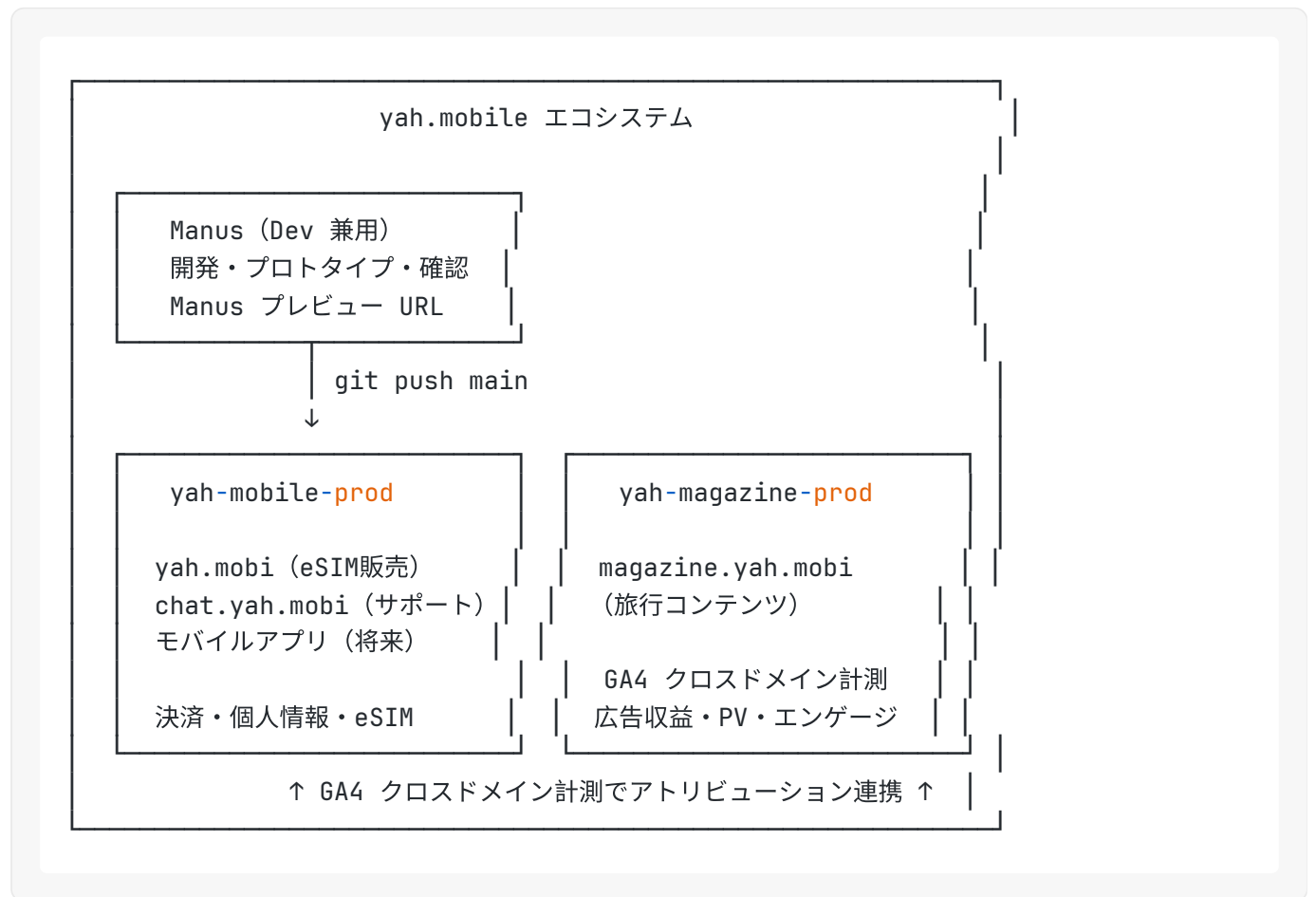
広告収益の最大化

- Firebase Analytics + GA4 のデータを Google Ad Manager に連携し、訪日旅行者向けの高単価広告枠を設定
- eSIM購入者と magazine 読者のオーディエンスを分離管理することで、広告ターゲティングの精度が向上する

yah.mobi との連携（クロスセル）

- magazine の記事内に yah.mobi の eSIM購入 CTA を自然に配置
- magazine 読者が yah.mobi で購入した場合、Firebase Analytics でアトリビューション（どの記事経由で購入したか）を計測できる
- ただしユーザーデータは GCP プロジェクトをまたいで自動共有されない。クロスセルのトラッキングは GA4 のクロスドメイン計測機能を使う

13.6 プロジェクト間の関係図



13.7 今から決めておくべきこと

決定事項	内容	タイミング
GCP プロジェクト名の確定	yah-mobile-prod / yah-magazine-prod (Dev は Manus が担うため不要)	Firebase 移行開始前
GA4 アカウント構成	1つの GA4 アカウント配下に2プロパティ (mobile-prod / magazine-prod)	Firebase 移行開始前
magazine のドメイン確認	magazine.yah.mobi の DNS が yah.mobi ドメインで管理されているか確認	今すぐ
クロスドメイン計測の設計	magazine → yah.mobi の購買アトリビュションをどう計測するか	magazine 開発開始前

[13] Google, “Firebase Projects Overview,” <https://firebase.google.com/docs/projects/learn-more>

[14] Google, “GA4 Cross-domain measurement,” <https://support.google.com/analytics/answer/10071811>